

CoderCommands MCP System

Step 03 Record — Ingest / Normalization Module

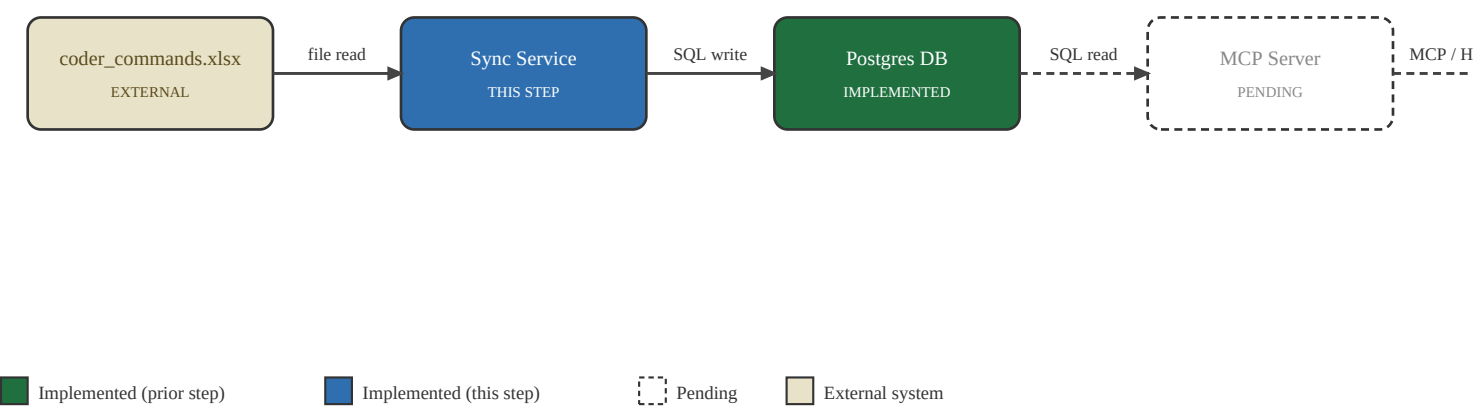
DOCUMENT	CoderCommands MCP System — Step Record
SYSTEM	CoderCommands MCP Command-Reference Server
STEP	03 — Ingest / Normalization Module
DATE	2026-08-15
REVISION	1.0
STATUS	Complete

1. Purpose & Scope

Build the transform stage that converts the 14 raw sheets into a flat stream of rows matching the Step 2 schema. This module owns all sheet-shape logic in one place so the write path (Step 4) and every downstream consumer only ever see one consistent shape, the same way a signal-conditioning stage isolates noisy field instrumentation from the rest of the control loop.

2. System Context Diagram

Signal-flow view of the four functional units (Sync Service, Postgres, MCP Server, FastAPI Gateway) plus the two external endpoints (source workbook, Claude Code client). Fill/border indicates build state as of this step.



3. File / Component Register

Role of every file or component introduced or modified in this step, with its upstream and downstream interfaces (I/O-list style).

Ref	File / Component	Role	Interface In	Interface Out
R1	sync/ingest.py	Pure parsing/normalization: reads the xlsx and returns NormalizedRow objects per sheet. Holds no DB connection and performs no I/O beyond reading the workbook.	coder_commands.xlsx	watcher.py (Step 4, in-process call)

4. Work Performed

- Header detection: 6 of 14 sheets (mongo, tmux, linux, docker, shell, postgres) start with a COMMAND/... header row and are auto-skipped; the other 8 have no header and are read from row 1.
- Section-divider detection: a row with exactly one populated, all-caps, non-command-like cell (e.g. 'IMAGE MANAGEMENT', 'TERMINAL OPERATIONS') is treated as a label applied to every following row until the next divider, not as a command of its own.
- Outline-mode detection: sheets where more than 30% of rows have a blank first column (jython, at 75%) are parsed as a two-level outline -- the populated-column-A row is both a command and a heading, and its blank-column-A children are column-shifted sub-entries tagged with that heading as their section.
- Continuation folding: the same blank-first-column pattern in sheets below the 30% threshold (shell, git, python, uPython -- a handful of rows each) is handled differently: appended as extra text onto the previous row rather than parsed as a new command, since inspection showed this text is wrapped prose, not shifted syntax.
- Ran the module against the live coder_commands.xlsx end-to-end: 608 normalized rows produced across all 14 topics.

5. Issues Encountered & Resolution

Issue	Root Cause	Resolution
First run crashed with AttributeError: 'NoneType' object has no attribute 'strip' during header detection.	The header check ran on raw cell values before the None-to-empty-string cleaning step, so any sheet with a blank leading cell crashed immediately.	Reordered the pipeline to clean every cell up front, then run header/divider classification only on the cleaned rows.
jython dropped from 91 raw rows to 19 normalized rows on the first structurally-working pass -- a 79% data loss that only surfaced by comparing row counts sheet-by-sheet.	jython uses a two-level outline (category row + blank-column-A sub-entries) that no other sheet uses at this density. The original logic treated any row with an empty first cell as empty and discarded it, silently dropping every sub-entry.	Added outline-mode detection keyed on the proportion of blank-first-column rows (>30%), with column-shifted parsing for sheets in that mode. jython recovered to 87 rows.
Applying the same column-shift fix to ALL blank-first-column rows would have fabricated nonsense commands elsewhere -- e.g. python's [None, "python doesn't do implicit conversions..."] is prose continuing the row above, not a new command called "python doesn't do implicit conversions".	shell, git, python, and uPython each contain a small number of blank-first-column rows that are wrapped explanatory text, structurally different from jython's outline despite looking identical at the single-row level.	Used the 30% threshold as a per-sheet mode switch: below it, blank-first-column rows are appended to the previous row's extra field instead of becoming their own row.

Issue	Root Cause	Resolution
Source data contains at least one corrupted character (jython 'keywords' row reads 'for█in' instead of the intended text, likely a special character that didn't survive a copy-paste into Excel).	Pre-existing data-quality issue in the workbook itself, not something introduced by ingest.	Left as-is rather than guessing the intended character -- ingest passes source text through unmodified. Flagged here for optional correction at the source next time that sheet is edited.

6. Verification Record

Check	Method	Result
Module runs end-to-end against the live workbook	python sync/ingest.py coder_commands.xlsx	PASS
Row count recovered after outline-mode fix (was 19, expected ~87)	Sheet-by-sheet row count comparison against raw XML dump	PASS
Total normalized rows across all 14 topics	608 rows (up from a first-draft 540 that still had the jython bug)	PASS
docker section divider ('IMAGE MANAGEMENT') correctly attributed to following rows	Manual inspection of normalized output	PASS
python stray continuation rows folded into previous row's extra, not spawned as new rows	Manual inspection of normalized output	PASS

7. Next Step / Open Dependencies

Step 4 wraps this module in a watcher that polls the xlsx for changes and performs the transactional per-topic write into Postgres, turning this pure function into a running sync service.